

Datenanalyse mit R mosaic

Karsten Lübke
2023-08-22

Vorbemerkungen

- R unterscheidet zwischen Groß- und Kleinbuchstaben
- R verwendet den Punkt . als Dezimaltrennzeichen
- Fehlende Werte werden in R durch NA kodiert
- Eine Ergebnisuweisung erfolgt über <-
- Übergabe |>
- Hilfe zur Funktion foo: ?foo

Innerhalb von mosaic:

```
analysiere(y ~ x | z , data = Daten)
```

d. h., modelliere y in Abhängigkeit von x getrennt bzw. bedingt für z aus der Datentabelle Daten.¹

Zusatzpakete müssen vor der ersten Benutzung einmalig installiert und geladen werden:

```
# Einmalig installieren
install.packages("mosaic", type = "binary")
# Laden, einmalig in jeder Sitzung
library(mosaic)
```

Daten

Einlesen:

```
getwd() # Aktuelles Arbeitsverzeichnis
# csv Datentabelle einlesen
Daten <- read.csv2("Pfad/Datei")
# xls Datentabelle einlesen
library(readxl) # Paket zum xls Import
Daten <- read_excel("Pfad/Datei")
```

Datenhandling:

```
filter()      # Beobachtungen filtern
select()      # Variablen wählen
mutate()      # Variablen verändern/ erzeugen
case_when()   # Fallunterscheidung
%>% ; |>      # Übergabe von Ergebnissen
```

Logik:

```
== ; !=      # Gleichheit bzw. Ungleichheit
> ; >= ; <= ; < # größer bzw. kleiner (gleich)
& ; |        # und bzw. oder
```

Arithmetik:

```
+ ; - ; * ; / # Grundrechenarten
^ ; sqrt(x)   # Potenz bzw. Quadratwurzel
exp(x) ; log(x) # e^x bzw. ln(x)
abs(x)        # Absolutbetrag
```

Datenanalyse

Grafische Verfahren:

```
gf_bar()      # Säulendiagramm
gf_histogram() # Histogramm
gf_boxplot()   # Boxplot
gf_point()     # Streudiagramm
mosaicplot()   # Mosaikplot (nicht ggformula)
```

Kennzahlen:

```
tally()      # Tabellierung, Häufigkeiten
prop()        # Anteile
count()       # Anzahl
diffprop()    # Differenz zweier Anteile
favstats()    # Kennzahlübersicht
sum()         # Summe
diffmean()    # Differenz zweier Mittelwerte
cor()         # Korrelationskoeffizient
pdata()       # Empirische Verteilungsfunktion
qdata()       # Quantilsfunktion
```

Verteilungen, Simulation

Normalverteilung:

```
xpnorm()      # Verteilungsfunktion Normalverteilung
xqnorm()      # Quantilsfunktion Normalverteilung
gf_qq()       # QQ-Plot (allgemein)
```

Randomisierung, Simulationen:

```
set.seed()    # Zufallszahlengenerator setzen
rflip()        # Münzwurf
do() *         # Wiederholung (Schleife)
sample()       # Stichprobe ohne Zurücklegen
resample()     # Stichprobe mit Zurücklegen
shuffle()      # Permutation
```

Modellierung

```
lm()           # Lineare Regression
glm(, family="binomial") # Logistische Regression
residuals()    # Residuen
fitted()       # Angepasste Werte
predict()      # Vorhersagen
```

Inferenz

```
prop.test()    # Binomialtest (approximativ)
xchisq.test()  # Chi-Quadrat Test
t.test()       # t-Test
aov()          # Varianzanalyse
```

¹Beim Mac ist ~ die Tastenkombination option+n, | die Tastenkombination option+7

Beispielanalyse

Vorbereitung:

```
library(mosaic)      # mosaic laden
data(KidsFeet)        # Interne Datentabelle laden
str(KidsFeet)         # Struktur der Datentabelle
?(KidsFeet)           # Hilfe, Info zur Datentabelle
```

Eine kategoriale Variable:

```
gf_bar(~ domhand, data = KidsFeet)
tally(~ domhand, data = KidsFeet)
prop(~ domhand, success = "L", data = KidsFeet)
```

Eine metrische Variable:

```
gf_histogram(~ length, data = KidsFeet)
favstats(~ length, data = KidsFeet)
```

Zwei kategoriale Variablen:

```
mosaicplot(domhand ~ biggerfoot, data = KidsFeet)
tally(biggerfoot ~ domhand, data = KidsFeet)
xchisq.test(biggerfoot ~ domhand, data = KidsFeet)
```

Zwei metrische Variablen:

```
gf_point(width ~ length, data = KidsFeet)
cor(width ~ length, data = KidsFeet)
cor.test(width ~ length, data = KidsFeet)
```

Zwei Stichproben, kategorial:

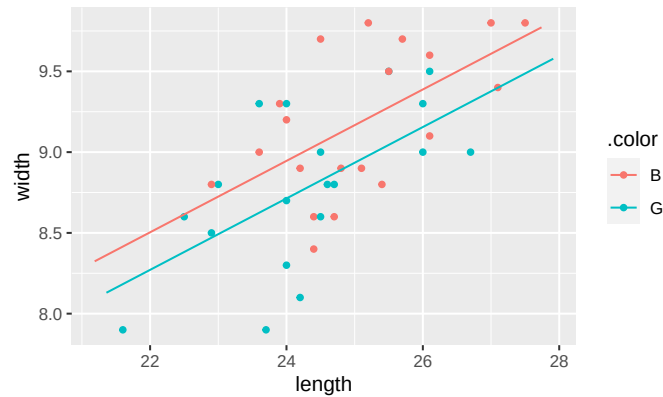
```
gf_bar(~ domhand | sex, data = KidsFeet)
prop(domhand ~ sex, success = "L",
     data = KidsFeet)
prop.test(domhand ~ sex, success = "L",
          data = KidsFeet)
```

Zwei Stichproben, metrisch:

```
gf_histogram(~ length | sex, data = KidsFeet)
gf_boxplot(length ~ sex, data = KidsFeet)
favstats(length ~ sex, data = KidsFeet)
t.test(length ~ sex, data = KidsFeet)
```

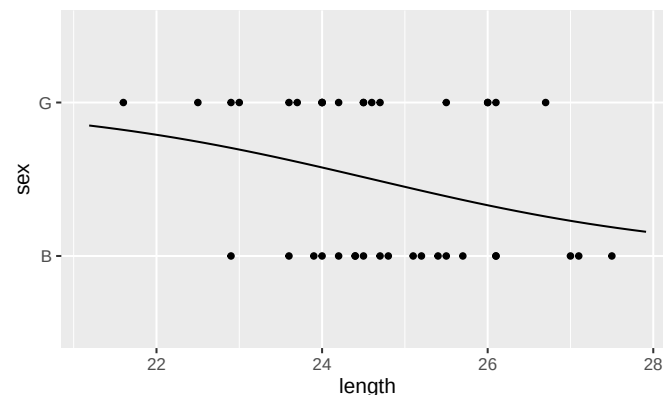
Lineare Regression:

```
erglm <- lm(width ~ length + sex,
             data = KidsFeet)
plotModel(erglm)
summary(erglm)
```



Logistische Regression:

```
ergglm <- glm(sex ~ length, family = binomial,
              data = KidsFeet)
plotModel(ergglm)
summary(ergglm)
# Odds Ratio
exp(coef(ergglm))
```

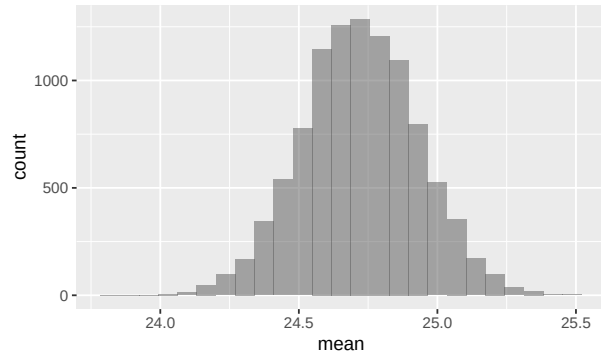


Datenhandling:

```
# Variablen selektieren
KidsFeet.length <- KidsFeet %>%
  select(length)
# Beobachtungen auswählen
KidsFeet.boys <- KidsFeet %>%
  filter(sex == "B")
# Variablen erzeugen
KidsFeet.in <- KidsFeet %>%
  mutate(length.in = 0.394*length)
# Bedingungen
KidsFeet.grouped <- KidsFeet %>%
  mutate(length.grouped = case_when(
    length > 25 ~ "large",
    length <= 25 ~ "not large"))
```

Bootstrap:²

```
set.seed(1896)
# Simuliere Ziehen zufälliger Stichprobe
Bootvtlg <- do(10000) *
  mean(~ length, data = resample(KidsFeet))
# Bootstrap-Verteilung
gf_histogram( ~ mean, data = Bootvtlg)
```



```
# Standardfehler
sd( ~ mean, data = Bootvtlg)
```

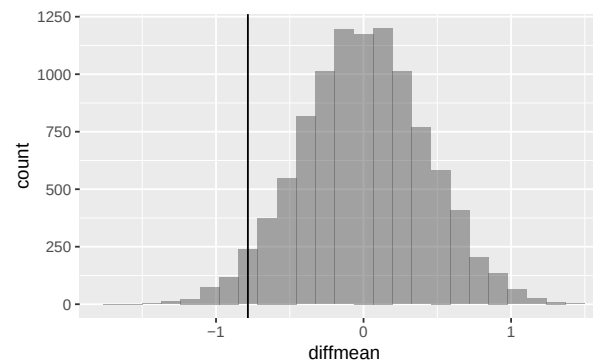
```
## [1] 0.2097938
```

```
# Bootstrap Konfidenzintervall
qdata( ~ mean, p = c(0.025, 0.975),
  data = Bootvtlg)
```

```
##      2.5%    97.5%
## 24.31019 25.13340
```

Permutationstest:

```
set.seed(1896)
# Beobachtete Differenz
est.diff <- diffmean(length ~ sex,
  data = KidsFeet)
# Simuliere zufällige Zuordnung:
Nullvtlg <- do(10000) *
  diffmean(length ~ shuffle(sex),
  data = KidsFeet)
# Verteilung unter H_0
gf_histogram( ~ diffmean, data = Nullvtlg) %>%
  gf_vline(xintercept = ~ est.diff)
```



```
# Permutationstest p-Wert
prop( ~ abs(diffmean) >= abs(est.diff),
  data = Nullvtlg)
```

```
## prop_TRUE
##      0.066
```

- Lizenz Creative Commons Attribution-ShareAlike 3.0 Unported.
- R Version: 4.3.1
- mosaic Version: 1.8.4.2

²Datentabelle hier sehr klein für Bootstrap Perzentile